

Same Task, Different Work: Prompt-Induced Waste in Coding Agents

A Preregistered Study of Reasoning, Tools, and Harnesses

Sarel Weinberger
PointFive
sarel.weinberger@pointfive.co

Amir Hozez
PointFive
amir@pointfive.co

August 2026 (v3)

Abstract

Two prompts can request the same code change and produce the same correct patch, yet cause a coding agent to perform radically different amounts of work. We test this effect in a preregistered benchmark spanning 4,644 valid runs, 24 deterministic tasks, six open-weight reasoning models, a post-registration Kimi-K3 replication, a first-party claude-sonnet-5 study, and two real agent harnesses. Asking an agent to develop and compare several approaches multiplies reasoning by $2.4\text{--}7.4\times$ across the frozen holdout without improving success. The instruction does not produce broader implementation: in 2,801 condition-blind annotated traces, it creates roughly three elaborated but discarded solution branches and exactly one implemented approach. Certainty language produces a different failure mode. It propagates into repeated tests, additional turns, latency, and context growth; runs with the highest observed redundant-verification level cost $18\times$ the clean-run median, execute $2.5\times$ the tool calls, and take $3\times$ the wall-clock, with no success gradient. These mechanisms therefore have different cost carriers: branch tournaments are primarily token-borne, whereas verification loops are tool-borne. Harness design amplifies both: in our matched setting, Claude Code carries a $12\text{--}15\times$ larger static prefix, uses $2\text{--}7\times$ more turns, and costs $5\text{--}30\times$ more per success than PI.DEV. The main effects survive a frozen holdout, paraphrase tests, Kimi-K3, and a first-party Sonnet 5 study. A bounded-efficiency instruction preserves diagnosis and validation while avoiding the measured waste mechanisms. Prompt engineering for coding agents is therefore not merely wording optimization: it is the design of what work the agent is asked to perform, how that work propagates through tools, and when it should stop.

1 Introduction: the same task can imply different work

A coding prompt does more than describe an objective. It also tells the agent what work to perform on the way there. Consider two instructions attached to the same small bug fix. One asks the agent to develop several distinct approaches, compare their trade-offs, and only then choose the best. The other asks it to inspect only what the evidence requires, make the smallest sufficient change, run the relevant tests, and stop. Both can produce the same correct patch. They do not produce the same computation. The first instruction launches a tournament of candidate solutions; the second directs the agent toward a bounded execution path.

This distinction is easy to miss when prompt efficiency is treated as a text-compression problem. In an agent, the user’s words sit inside a much larger system: a fixed harness prefix, tool schemas, repeated model turns, tool outputs that re-enter context, and provider caching. A short sentence

Instruction family	Selected observed effect	Measured success gain
Multiple approaches	2.4–7.4× reasoning on all six holdout models; approximately three discarded solution branches	None
Deep thinking	1.6–2.2× reasoning where selected; roughly 2.2× recorded text with no new observable functional units	None
Maximum certainty	More re-verification, repeated tests, and post-completion activity	None
Verbose repetition	Approximately 1.0× reasoning despite a much longer user prompt	None
Bounded efficiency	At or below baseline reasoning, with diagnosis and final validation preserved	Preserved

Table 1: The selected effects that organize the paper. Full per-model holdout results, confidence intervals, selection rules, and stress-family results appear in Appendix C.

can therefore be expensive if it requests expensive behavior, while a long restatement can be nearly free if it changes nothing about the requested work.

We ask three questions. First, how much additional work can prompt wording create when the task, model, harness, and success criterion are held fixed? Second, what kind of work is created—discarded reasoning branches, re-verification, repository exploration, or tool use? Third, when does that work remain inside model deliberation, and when does it propagate into test runs, additional turns, latency, and end-to-end cost?

We answer these questions with a preregistered paired benchmark of 24 coding tasks with hidden deterministic evaluators. The original study uses six open-weight reasoning models and two harnesses, PI.DEV and Claude Code; post-registration studies add Kimi-K3 and first-party claude-sonnet-5. The full corpus contains 4,644 valid runs, and 2,801 provider-exposed reasoning traces receive condition-blind, evidence-quoted semantic annotation. The technical design, exact prompts, full matrices, instrumentation checks, and robustness analyses are reported in the appendices. The main text follows the argument rather than the collection chronology.

The evidence supports three claims. **Prompt wording allocates agent work:** requesting compared approaches or repeated certainty creates large cost increases without a measured success gain, whereas a length-matched restatement does not. **Waste has structure:** solution tournaments produce token-borne discarded branches, while certainty pressure produces tool-borne verification loops. **The harness amplifies the prompt:** prefix size, turn discipline, and tool policy determine how often the consequences of an instruction are replayed.

2 Prompt wording allocates agent work

The primary conditions preserve the task objective, acceptance criteria, and test command; only the added instruction changes. The strongest result is not that longer prompts cost more. They generally do not. It is that phrases which request additional work reliably cause the agent to perform that work, even when the benchmark rewards only the final correct patch.

On the frozen eight-task holdout, `multiple_approaches` is classified wasteful on every one of the six original models, increasing reasoning by 2.4–7.4× with no material success improvement. `deep_thinking` increases reasoning by 1.6–2.2× on every model for which it was selected. By

contrast, verbose repetition of the same objective, scope, acceptance criteria, and stop condition remains approximately $1.0\times$ across models. The contrast identifies the relevant unit: not prompt length, but the work implied by the instruction.

The negative controls matter. A bounded-efficiency instruction—begin with the failing test and likely implementation files, inspect more only when evidence requires it, make the smallest sufficient change, run the relevant tests, and stop when the acceptance criteria pass—is neutral or better on all six holdout models. It does not reduce observed diagnosis, evidence collection, or final validation. Its role is not to make the model terse at any cost; it declines to authorize work that the task does not require.

The stress family sharpens the same distinction. Irrelevant context and conflicting constraints are nearly free in this benchmark, but a plausible wrong architectural hint raises reasoning $2.61\times$, and ambiguous scope produces both the lowest success rate (83%) and $1.44\times$ reasoning. The models filter generic noise better than they resist plausible misdirection. Detailed stress results appear in Appendix C.

3 Waste has structure

The aggregate token ratios tell us that work increased, but not what the agent did with it. The annotated traces and deterministic tool telemetry show that the two dominant mechanisms are qualitatively different.

3.1 Branch tournaments: expensive ideas that never reach the repository

The `multiple_approaches` instruction does exactly what it says. Relative to baseline, the model considers 3.5 additional approaches on average, elaborates about three that are never used, and implements exactly one. The frozen holdout reproduces the same $+3.5/ +3.0/ +1.0$ pattern. The instruction therefore does not buy a second implementation or an experimental comparison; it buys an internal tournament whose losing branches are discarded before they reach the repository.

This mechanism is primarily token-borne. The first observed discarded branch raises median no-cache run cost by approximately $1.9\times$, while median tool calls remain nearly flat (7 to 8). Higher branch levels do not produce a corresponding rise in repository exploration or code edits. The agent is not broadly testing alternative implementations; it is elaborating them in reasoning text and then abandoning them.

3.2 Verification loops: when caution becomes action

Certainty pressure follows another path. The `max_certainty` instruction adds re-verification of already established facts, more reasoning devoted to checking, repeated tests, and activity after the last edit and first fully green visible test. Every additional tool result re-enters the model context, where it can induce another turn and another round of checking.

Figure 1 shows the contrast. Branch count changes model cost while tool activity stays nearly flat. Redundant verification causes cost, calls, and latency to rise together. At the highest observed verification level, runs cost $18.25\times$ the clean-run median, execute 15 rather than 6 median tool calls, and take $3\times$ the wall-clock, with no success gradient. This is a descriptive comparison across observed mechanism levels, not a claim that the annotation level itself was randomly assigned; the causal result is that certainty wording increases the verification behavior and downstream work.

Waste mechanisms have different cost carriers (judge level vs deterministic telemetry; success flat across levels)

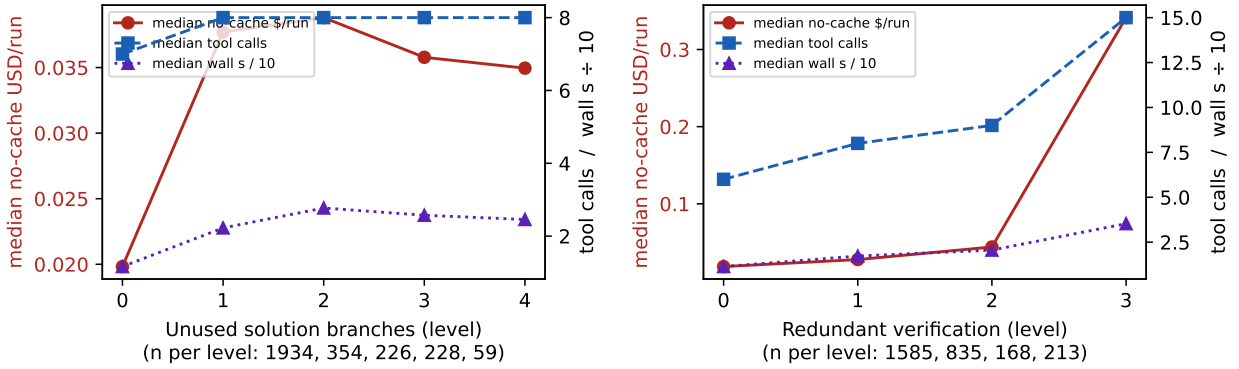


Figure 1: Different mechanisms have different cost carriers. Left: the first discarded solution branch nearly doubles median no-cache run cost while tool calls stay flat. Right: redundant verification escalates tool calls, latency, and cost together. Success is flat across the displayed levels. Full counts and robustness statistics appear in Appendix C.

3.3 Misdirection: plausible hints are more dangerous than noise

The wrong-hint condition adds one unsupported assumption and one stated hypothesis while grounded hypotheses remain unchanged. Deterministically, pre-first-edit deliberation rises $4.2\times$. Unsupported assumptions are also the only annotated semantic marker negatively associated with success ($\rho = -0.19$). This mechanism is neither a branch tournament nor a verification loop: the model is spending work to make an unverified premise coherent.

3.4 Bounded efficiency: avoiding waste without suppressing work

Bounded efficiency is the useful null. Relative to baseline, observed error diagnosis, final validation, and unused branches all change by 0.0; every deterministic redundancy metric also changes by 0.00, with tests, edits, and diagnosis preserved. The instruction is effective not because it removes a necessary reasoning stage, but because it does not trigger the additional mechanisms described above.

Visible traces are not a complete transcript of model computation. Consequential computation may occur without a semantically interpretable chain of thought [7]. We therefore make claims about provider-exposed reasoning text and observable agent behavior, not hidden cognition. claude-sonnet-5, whose reasoning is billed inside output but not recoverable as text, is excluded from all trace-level claims.

4 The harness amplifies the prompt

A prompt does not determine cost alone. The harness determines the fixed context around it, how many turns occur, which tools are exposed, and how often the consequences of an instruction are replayed.

For a trivial user request, PI.DEV transmits 1,147–1,642 prefix tokens, while Claude Code transmits 15,983–20,330—a $12\text{--}15\times$ gap. On matched model–task–prompt cells, Claude Code also uses $2\text{--}7\times$ more turns. Together these mechanics produce a $5\text{--}30\times$ difference in cost per success

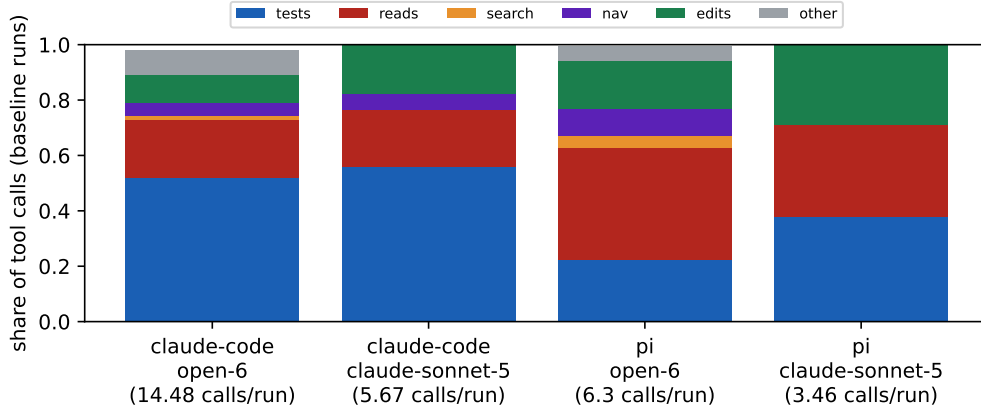


Figure 2: Baseline tool-call composition. Claude Code is verification-heavy; PI.DEV is inspection-heavy. The complete model and run-count breakdown appears in Appendix B.

in the original open-model comparison. The result persists on the frontier first-party path: native Claude Code with claude-sonnet-5 costs about $15\times$ the PI.DEV baseline per compliant success on the same tasks and no-cache accounting.

The two harnesses also spend their calls differently. Half of Claude Code tool calls are test executions (52%), versus 22% under PI.DEV; PI.DEV is more inspection-heavy, with 40% reads versus 21% under Claude Code. These are not merely billing differences. A verification-heavy harness gives certainty language more room to propagate into repeated actions.

Caching changes the bill but not the work. Automatic provider-side prefix caching rebates roughly 61% of the would-be original-study bill and 69–75% on the first-party Sonnet 5 study, while leaving every behavioral metric unchanged. We therefore report actual cost and estimated no-cache cost separately: the former describes what was charged under a particular cache state; the latter better reflects the logical work transmitted. The dual-side wire-capture, gateway validation, metadata-loss checks, cache anomalies, and native billing reconciliation are documented in Appendix B.

5 The effect is not tied to one model or one phrase

The central result survives four distinct tests of scope.

Frozen holdout. The main prompt families were selected on the development split and then rerun on eight unseen tasks with five repetitions per cell. Multiple approaches remains wasteful on all six models, deep thinking replicates wherever selected, and bounded efficiency remains neutral or better.

First-party frontier model. Against the native Anthropic API, `multiple_approaches` raises total output tokens $2.70\times$ under PI.DEV and $2.52\times$ under native Claude Code. `max_certainty` is the largest native-Claude Code family at $4.13\times$ output tokens and $2.7\times$ no-cache cost per success. `deep_thinking` is milder ($1.25\text{--}1.30\times$), consistent with the possibility that adaptive thinking absorbs some explicit thinking cues; the internal mechanism is not directly observable.

A frugal-by-default reasoner. Kimi-K3 begins from a much lower reasoning floor (median 55 tokens versus 351/195 for the K2 comparators). Against its own baseline, multiple approaches reaches $16.6\times$ and deep thinking $14.8\times$. The large ratios are partly a denominator effect: absolute waste tokens remain at or below K2-generation levels, and Kimi-K3 still has the lowest baseline cost per compliant success. The replication shows why relative inflation and absolute spend must be reported together.

Paraphrases and repetition. All four paraphrases of multiple approaches, deep thinking, and maximum certainty elevate reasoning on every model tested, while length-matched controls remain at $0.96\text{--}1.05\times$. All five bounded-efficiency formulations remain at or below baseline ($0.85\text{--}0.93\times$). The phenomenon is therefore semantic, but its magnitude remains lexical: multiple-approaches formulations range from $3.0\times$ to $6.7\times$, and instruction position shifts the effect modestly. No direction reversal was observed across repeated cells for a confirmed waste variant; the only reversals occur in conditions originally classified as neutral. The complete protocols, run matrices, confidence intervals, and registered expansion status appear in Appendix E.

6 Implications: prompt design is work design

The findings suggest four practical principles.

Ask for alternatives only when alternatives are the deliverable. A request to compare approaches is appropriate when comparison itself is valuable. For an ordinary patch, it reliably buys discarded deliberation. When alternatives are needed, the prompt should state whether they must be briefly listed, deeply analyzed, implemented, or experimentally evaluated. Those are different workloads.

Replace certainty language with an executable stop rule. “Be absolutely certain” has no observable stopping condition and can convert caution into a verification loop. A measurable instruction is safer: run the relevant suite after the final edit; rerun only after a relevant change or a failure; stop when the acceptance criteria pass.

Optimize the mechanism, not only the token count. Branch tournaments are primarily token-borne, so they require constraints on elaboration and deliverables. Verification loops are tool-borne, so they require stopping policies and duplicate-action guards. Context compression can reduce the price of tool outputs, but it does not by itself prevent the agent from issuing unnecessary calls.

Measure harness behavior separately from model behavior. Static prefix, turn count, tool composition, cache state, and protocol translation can dominate the user-prompt effect. Benchmarks that do not capture both sides of a gateway can silently attribute harness failures or metadata loss to the model.

7 Related work

Prior work studies prompt sensitivity, overthinking and test-time compute, chain-of-thought faithfulness, cost-aware planning, tool selection, and context reduction. When2Tool asks when calls are

necessary [8]; CostBench and CATP-LLM study cost-aware plans [9, 10]; recent surveys organize efficiency levers across memory, tools, and planning [11]; and CORVUS reduces the context cost of coding-agent observations [12]. Our narrower contribution is to hold the task and agent configuration fixed and trace a user instruction through observable reasoning structure, downstream tool behavior, and end-to-end cost. We claim no priority beyond the reviewed scope.

8 Limitations

First, provider-exposed reasoning text is neither complete nor necessarily faithful; our semantic claims concern observable traces, not hidden cognition. Second, the tasks are small (at most four files) and have high success ceilings. Multiple approaches may be useful on repository-scale or architectural work where alternatives require genuine implementation and evaluation. Third, the prompt conditions are randomly assigned within paired blocks, but annotated mechanism levels are observed mediators rather than randomized treatments; the $18\times$ verification comparison is therefore descriptive. Fourth, exact per-turn cost attribution is not available, so tool-induced model cost is bounded at 4–12% rather than identified exactly. Fifth, the Claude Code arms are smaller, provider prices and cache behavior are time-specific, and the remaining new-task and long-validation expansion arms were still in progress at writing. Full measurement limitations, judge agreement, zero-inflated count handling, and classifier caveats appear in Appendix D.

9 Conclusion

Two prompts can produce the same correct patch while asking the agent to perform radically different amounts and kinds of work. Requests for several approaches create internal solution tournaments whose losing branches are paid for and discarded. Demands for certainty create verification loops that escape into tools, turns, latency, and context growth. Harness design then determines how strongly those consequences are amplified. Across the tested tasks, none of these mechanisms improves success, while a bounded instruction preserves diagnosis and final validation. Prompt engineering for coding agents should therefore be understood as work design: specify the deliverable, constrain unnecessary paths, and define when the agent is done.

Acknowledgments

Experiments, infrastructure, and analysis were carried out with Claude (Anthropic) operating as an autonomous research assistant under the authors’ direction and budget approvals.

References

- [1] PointFive Labs. *prompt-efficiency-benchmark: benchmark code, task fixtures, prompt variants, raw ledgers, and analysis*. <https://github.com/PointFiveLabs/prompt-efficiency-benchmark>, 2026.
- [2] Earendil Works. *Pi: an AI agent toolkit and coding-agent CLI*. <https://pi.dev>, 2026.
- [3] Anthropic. *Claude Code*. <https://claude.com/claude-code>, 2026.
- [4] Together AI. *Inference platform and model catalog*. <https://www.together.ai>, 2026.

- [5] BerriAI. *LiteLLM proxy (v1.93.0)*. <https://github.com/BerriAI/litellm>, 2026.
- [6] Moonshot AI. *Kimi K3 technical blog*. <https://www.kimi.com/blog/kimi-k3>, 2026.
- [7] V. Baherwani, T. Goldstein, and A. Panda. *Not All LLM Reasoning Is Visible in the Chain-of-Thought*. arXiv:2607.22925, 2026.
- [8] C.-E. Sun, L. Liu, G. Yan, Z. Wang, and T.-W. Weng. *LLM Agents Already Know When to Call Tools — Even Without Reasoning (When2Tool)*. arXiv:2605.09252, 2026.
- [9] J. Liu, C. Qian, Z. Su, Q. Zong, S. Huang, B. He, and Y. R. Fung. *CostBench: Evaluating Multi-Turn Cost-Optimal Planning and Adaptation in Dynamic Environments for LLM Tool-Use Agents*. arXiv:2511.02734, 2025.
- [10] D. Wu, J. Wang, Y. Meng, Y. Zhang, L. Sun, and Z. Wang. *CATP-LLM: Empowering Large Language Models for Cost-Aware Tool Planning*. arXiv:2411.16313, 2024.
- [11] X. Yang, L. Li, H. Zhou, T. Zhu, X. Qu, Y. Fan, Q. Wei, R. Ye, L. Kang, Y. Qin, D. Liu, Q. Li, N. Ding, S. Chen, and J. Shao. *Toward Efficient Agents: Memory, Tool Learning, and Planning*. arXiv:2601.14192, 2026.
- [12] M. Zheng, D. O’Brien, S. Cui, P. Pashakhanloo, R. Mukherjee, M. Kim, and S. Kuhar. *CORVUS: Context Optimization and Reduction Via Underlying Synchronization for LLM Coding Agents*. arXiv:2607.22711, 2026.

A Experimental Design and Preregistration

A.1 Preregistration

Hypotheses H1–H8 and H12–H17, metric definitions, waste-classification rules, and experiment protocols were frozen in the public repository before any benchmark result was inspected. The holdout phase additionally froze per-model variant selections, prompts, evaluators, and analysis code before any holdout run. The Kimi-K3 replication froze its task rule, matrix, and material-difference thresholds before its first result. Expansion hypotheses E1–E12, manifests, power analysis, budget caps, and stopping rules were registered separately; expansion runs remain tagged separately and are not merged into the original preregistered results.

A.2 Models, harnesses, and serving

The original study froze six reasoning models ($\geq 500\text{B}$ -class) served by Together AI: DeepSeek-V4-Pro, Kimi-K2.6, Kimi-K2.7-Code, Nemotron-3-Ultra-550B-A55B, Inkling, and GLM-5.2. Two harnesses operated the models: PI.DEV 0.82.1 through OpenAI chat completions directly, and Claude Code 2.1.220 through the Anthropic Messages protocol and a pinned LiteLLM 1.93.0 translation gateway. Two logging reverse proxies captured both sides of the gateway with secrets redacted, preserving the provider’s raw usage object for every run. Kimi-K3 and claude-sonnet-5 were added only through separately frozen post-registration studies.

A.3 Tasks and evaluation

The benchmark contains 24 deterministic coding tasks: eight low, eight medium, and eight high complexity; nine Python, nine JavaScript, and six Go. Each task has a clean fixture, one concrete

objective, explicit acceptance criteria, visible tests, hidden deterministic tests that never exist in the workspace during a run, allowed and forbidden paths, and a per-task evaluator. Sixteen tasks form the development split and eight were frozen as holdout. Every fixture was verified to fail its visible tests before the agent’s fix. The small task size (≤ 4 files) is deliberate for causal control and is also a limitation on production-scale generalization.

A.4 Prompt conditions

A generator renders 18 frozen variants per task. Nine primary variants preserve the objective, acceptance criteria, and test command verbatim: `baseline`, `verbose_repetition`, `deep_thinking`, `exhaustive_exploration`, `multiple_approaches`, `max_certainty`, `adjacent_cleanup`, `no_questions_autonomy`, and `bounded_efficiency`. `goal_only` and `scoped_authorization` define additional harness axes. Seven stress variants intentionally break semantic equivalence through ambiguity, conflicting constraints, irrelevant context, misleading hints, missing criteria, or multi-turn splitting and restatement; they are analyzed separately.

Because wording is the manipulated variable, lexical form, order, length, and position are part of the experimental treatment. Table 2 reproduces the principal frozen clauses. Appendix F contains all complete templates, representative rendered prompts, multi-turn boundaries, and SHA-256 checks against the run ledger.

A.5 Metrics and classification rules

Behavioral outcomes include provider-reported reasoning tokens, visible output tokens, tool calls by type, turns, files inspected, duplicate reads, repeated searches, repeated test runs, time to first edit, wall time, out-of-scope changes, and success on visible plus hidden tests. Billing outcomes include uncached and cached input, logical input, actual cost at pinned catalog prices, and estimated no-cache cost.

The original primary outcome is the paired reasoning ratio: each run’s reasoning tokens divided by the median baseline reasoning in the same model–harness–task block. A variant is classified wasteful only when the median ratio exceeds 1.5, the task-clustered 95% confidence interval has a lower bound above 1.1, there is no material success gain, and the effect appears on multiple tasks. Count outcomes use per-task paired medians, task-clustered bootstrap intervals, and sign-flip permutation tests rather than pooled correlations alone.

B Instrumentation, Gateway, Cache, and Billing

B.1 Fixed prefix and turn overhead

For a trivial prompt, PI.DEV transmits 1,147–1,642 tokens of fixed prefix (system prompt plus four tool schemas), depending on tokenizer. Claude Code transmits 15,983–20,330 tokens (approximately 6.7 KB of system prompt plus 24 tool schemas), a 12–15 $\times$ difference. The user prompt is under 5% of logical input under PI.DEV and under 1% under Claude Code. Consequently, prompt length is a poor economic proxy even though prompt content can strongly change downstream work.

B.2 Gateway validation and metadata integrity

Three gateway routes were attempted. A Responses-API route passed only three of six models: GLM-5.2, DeepSeek-V4-Pro, and Inkling returned empty continuations after tool results when reasoning items were not replayed. A chat-completions route with default settings passed zero

Condition	Exact manipulated clause	Intended manipulation
baseline	<code>{OBJECTIVE} Scope: {SCOPE} Acceptance criteria: {ACCEPTANCE_CRITERIA} Verify with: {TEST_COMMAND} Stop when the tests pass.</code>	precise control (objective, scope, criteria, test command, stop condition)
multiple_approaches	<code>Develop several distinct approaches to this fix, compare their trade-offs explicitly, and only then select and implement the best one.</code>	force an internal solution tournament
deep_thinking	<code>Think very deeply about this. Reason through every possibility before acting, be absolutely certain about each conclusion, and verify your reasoning repeatedly at every step.</code>	demand deep, repeatedly verified reasoning
exhaustive_exploration	<code>Before editing anything, inspect the entire repository and make sure you understand the complete system: read every source file and understand how all the parts connect, so your change is informed by the whole codebase.</code>	demand whole-repository understanding first
max_certainty	<code>You must be absolutely confident in the result. Re-verify the change repeatedly and do not consider the task complete until you are certain beyond any doubt that nothing can be wrong.</code>	demand certainty beyond doubt / re-verification
adjacent_cleanup	<code>While you are in there, feel free to clean up anything adjacent that could be improved: refactor awkward code, tidy naming, and make related general improvements where you see the opportunity.</code>	authorize scope expansion
no_questions_autonomy	<code>Do not ask me any questions. Infer any missing details yourself and do whatever is necessary to complete the task autonomously.</code>	forbid questions; force autonomy
bounded_efficiency	<code>Work efficiently: begin with the failing test and the most likely implementation files; inspect additional files only when evidence requires it; avoid unrelated cleanup; make the smallest sufficient change; run the relevant tests; stop as soon as the acceptance criteria pass.</code>	explicit scope + smallest change + stop rule
verbose_repetition	<code>To restate the requirements so nothing is missed: the change must stay within the scope given above (shop/discount.py only), it must satisfy the same acceptance criteria already listed, and the verification command remains <code>python3 -m unittest discover -s tests -t . -v</code>. In other words, do the task exactly as described, within exactly the same boundaries, checked by exactly the same tests, and finish under exactly the same stop condition as stated before.</code>	restate identical requirements (length control)
goal_only	<code>(baseline reduced to {OBJECTIVE} only; scope, criteria, test command, stop condition removed)</code>	strip everything but the objective
scoped_authorization	<code>You are authorized to modify only: shop/discount.py only You are NOT authorized to modify anything else; treat all other paths as read-only. [replaces:] Scope: shop/discount.py only</code>	recast scope as explicit authorization
misleading_architecture	<code>Note: we believe the problem is caused by the test fixtures in tests/ --- start there. [replaces:] Scope: shop/discount.py only</code>	inject a wrong architectural hint (stress)

Table 2: Principal prompt conditions with the exact frozen wording (byte-exact; SHA-256-verified against the run ledger). Exact manipulated clause shown; complete rendered templates appear in Appendix F. Placeholders are instantiated per task, identically across variants of the same task.

of six because the gateway silently dropped all 24 tool schemas when function-calling support was not declared per model. The final route—function calling declared and parameter dropping disabled—passed all six models on a 14-capability suite including multi-step tool loops. A text response alone was therefore not treated as gateway validation.

Upstream, all open models report explicit reasoning-token and cached-token fields. The gateway preserves total input and output but drops reasoning detail and re-derives cache fields. The Claude Code-side cost report also uses the harness price table for aliased models and was measured at $3.5\times$ the provider-side cost. All analyses therefore use provider-side capture.

B.3 Cache behavior

Provider-side prefix caching is automatic and free to populate in the original serving route. It rebates approximately 61% of the would-be bill (billed cost approximately 39% of no-cache cost) while leaving behavioral metrics unchanged. Hits are probabilistic: full, partial, and no hit can alternate under identical conditions. The cache is shared across sessions; fresh Claude Code sessions routinely receive first-turn hits covering roughly 90% of the 16–20k prefix. A 60-second delay showed no eviction, while changing the working directory breaks reuse because the directory is embedded in the system prompt. Nemotron through PI.DEV received zero cached tokens throughout all phases for three days, a provider-side anomaly that would distort comparisons based only on billed cost.

The first-party claude-sonnet-5 study received cache rebates of 69–75%. Because caching changes price without changing agent behavior, the paper keeps behavioral work, actual billed cost, and no-cache cost conceptually separate.

B.4 Native Claude Code billing reconciliation and classifiers

The $5\text{--}30\times$ open-model harness gap cannot be caused by Anthropic-side classifiers because those requests were served by Together AI through the translation gateway and never entered Anthropic’s model-serving path. The measured gap is accounted for by the larger prefix, $2\text{--}7\times$ more turns, and verification-heavy tool composition.

For the 54 native Claude Code runs against claude-sonnet-5, recorded cost reconciles from visible usage categories: uncached input, cache reads, cache writes, and output. The median discrepancy is zero and 53/54 runs reconcile within 1%. An initial uniform 33.3% discrepancy came from comparing standard with introductory pricing. Remaining differences imply cache-write rates whose median and 90th percentile equal the documented five-minute write premium ($1.25\times$ input rate), with a small upper tail consistent with one-hour-TTL writes ($2\times$ input rate) mixed within a run. The residual tracks cache-write token volume rather than request count or turns. No separately billed classifier cost is detectable. This is a claim about billing, not about whether provider-side classifiers execute or add latency.

B.5 Harness tool composition

At baseline, 52% of Claude Code tool calls are test executions versus 22% under PI.DEV. PI.DEV is inspection-dominated, with 40% reads versus 21% under Claude Code. The original open models average 14.48 calls per baseline run under Claude Code and 6.30 under PI.DEV; claude-sonnet-5 averages 5.67 under native Claude Code and 3.46 under PI.DEV. Extreme runs with 77–79 failed calls are Claude Code+GLM permission-friction loops distributed across prompt conditions, and are classified as harness-caused rather than prompt-caused.

C Full Primary Results and Cost Decomposition

C.1 Frozen holdout

For each model, the three most waste-inducing variants were selected using development data only, frozen, and rerun on eight unseen holdout tasks with five repetitions ($n = 40$ per selected cell under PI.DEV).

Model	mult. approaches	deep thinking	adj. cleanup	max certainty	exh. exploration	bounded efficiency
DeepSeek-V4-Pro	2.92 W	2.12 W	—	1.85 W	—	0.97 N
Kimi-K2.6	7.40 W	2.21 W	—	—	1.29 w	1.06 N
Kimi-K2.7-Code	5.98 W	1.86 W	—	—	—	0.98 N
Nemotron-3-Ultra	2.44 W	1.57 W	—	1.48 w	—	1.16 N
Inkling	5.08 W	—	3.13 W	—	1.39 n	1.04 N
GLM-5.2	6.18 W	2.18 W	4.25 W	—	—	0.48 N

Table 3: Median reasoning ratio versus each model’s own baseline on eight unseen tasks. W = confirmed wasteful; w = replicated but weaker; n = not replicated; N = confirmed neutral. A dash means that the feature was not among that model’s development-selected holdout conditions, not that it was tested and found absent.

Multiple approaches is confirmed wasteful on all six models. Deep thinking is confirmed on every model where selected and roughly doubles wall time with similar tool behavior. Adjacent cleanup is confirmed on Inkling and GLM-5.2; together with autonomy language, it is one of only two families that produce out-of-scope edits (5–8% of their runs, approximately zero elsewhere). Autonomy itself does not replicate a reasoning effect on the holdout ($1.00\times$) but retains the scope-violation effect. Verbose repetition remains approximately $1.0\times$.

C.2 Stress family

Against same-task screening baselines, median results across models are: misleading architecture, $2.61\times$ reasoning; ambiguous scope, 83% success and $1.44\times$ reasoning; one task split across two turns, approximately $1.31\times$; full restatement each turn, approximately $1.38\times$; irrelevant context, $1.03\times$; conflicting constraints, $1.05\times$. Stress variants intentionally break semantic equivalence and are not merged with the primary causal wording analysis.

C.3 Harness comparison

On matched model–task–prompt cells, both harnesses solve the pilot tasks at approximately 100%, but Claude Code costs 5–30 $\times$ more per success. For Nemotron the difference is approximately 18 $\times$. Claude Code uses 10–41 turns versus 5–7 under PI.DEV. Kimi-K2.7-Code remains near 10 Claude Code turns, while GLM-5.2, Kimi-K2.6, and Nemotron approach the turn ceiling on tasks they solve in at most seven turns under PI.DEV. Prompt effects interact with the harness: goal-only prompts reduce reasoning on four of five models under Claude Code, whose system prompt supplies methodology, while exploration cues amplify 4–4.8 $\times$ under Claude Code versus 1.1–2.4 $\times$ under PI.DEV on matched cells.

Redundant verification	n	median (rel.)	mean (rel.)	wins. mean (rel.)	95% CI	calls
level 0	1,585	1.00×	1.00×	1.00×	[0.96, 1.03]	6
level 1	835	1.48×	1.77×	1.70×	[1.44, 1.58]	8
level 2	168	2.36×	4.34×	4.28×	[2.01, 2.67]	9
level 3+	213	18.25×	17.74×	18.38×	[15.13, 21.06]	15

Table 4: Cost by condition-blind judge-annotated redundant-verification level. Costs are normalized to the level-0 median. Level 3+ also takes $3\times$ the wall-clock; success remains flat. The gradient survives winsorization at the 95th percentile, and the level-3+ lower quartile is $10\times$ the level-0 median.

C.4 Mechanism-level cost table

Unused branches behave differently. The first discarded branch raises median cost approximately $1.9\times$ ($n = 1,934$ clean and 354 one-branch runs) while calls remain 7 to 8, and the cost curve plateaus at higher branch levels. These mechanism-level comparisons are descriptive joins of annotation and telemetry; prompt-level paired effects provide the causal contrast.

C.5 Prompt-level tool effects

Across the six original models, `max_certainty` adds a median 1.75 post-success calls [1.0, 2.5], one test execution, and four seconds of latency; all six models share the direction. The worst observed loop is Kimi-K2.6 rerunning an already-green suite six times. `deep_thinking` leaves calls effectively unchanged ($+1.0$, $p = .26$), while its cost increase is significant ($p = .03$), supporting a token-borne interpretation. `multiple_approaches` changes code edits by 0.0 on all six models and has a directional tool-call delta of $+1.5$ ($p \approx .06$). `misleading_architecture` is also directional at the tool layer ($+1.75$ calls and $+0.5$ failed calls), while its robust effect is pre-edit reasoning. `bounded_efficiency` is a full tool-layer null: all redundancy metrics change by 0.00 [0, 0] ($p = 1.0$), with tests, edits, and diagnosis preserved.

C.6 Tool-induced model cost

Tools have no direct monetary charge in the benchmark, but their results re-enter context, add input tokens, alter cache accounting, and can induce further turns. Tool-induced model cost is bounded at 4–12% of run cost. Exact attribution is not possible because a turn’s input mixes tool results, assistant output, and harness scaffolding. Dollars, latency, and call counts are therefore reported separately; no direct tool dollar cost is fabricated.

D Semantic Annotation, Tool Extraction, and Measurement Limits

D.1 Reasoning-trace availability

A five-tier audit of run artifacts preceded the semantic analysis. Full provider-exposed reasoning text is available for the seven open-weight models, including Kimi-K3, under both harnesses. PI.DEV records turn events; Claude Code transcripts receive the provider reasoning field through the gateway, verifiable against raw wire captures. `claude-sonnet-5` has empty thinking blocks and bills reasoning inside output, so it is included in cost, success, turn, latency, and deterministic tool analyses but excluded from semantic annotation. No reasoning-unit estimate is inferred from its tool behavior.

Condition		Observable reasoning effect	Tool-layer effect	Cost carrier	Success	Evidence
multiple approaches	ap-	+3 discarded branches; +1 implemented	weak/directional ($p \approx .06$); edits +0	token-borne	none	strong semantic; directional tool
deep thinking		2.2× text; no new observable functional units	none ($p = .26$)	token-borne	none	strong
maximum certainty		+1 redundant re-verification on 6/6 models	+1.75 post-success calls; repeat tests	tool-borne plus induced tokens	none	strong
misleading hints		+1 unsupported assumption; 4.2× pre-edit deliberation	directional (+1.75 calls; +0.5 failed)	reasoning-borne	none / possible harm	mixed
bounded efficiency		no loss of diagnosis or validation	null ($p = 1.0$)	avoids measured waste	preserved	strong null

Table 5: Integrated mechanism summary. “None” means no measured success gain in the paired prompt comparison.

D.2 Condition-blind semantic judge

A frozen rubric defines 14 composition counts, six waste mechanisms, and ordinal quality scores (H-S1–H-S6). The judge never sees prompt text, variant names, or condition labels. Grammar-constrained JSON and mandatory evidence quotations yield 98.9% valid annotations; 91.1% of nonzero counts are evidence-backed. Researcher-blind hand labels agree within ± 1 on 84.9% of fields. Same-model prompt-sensitivity agreement is $\kappa = 0.44$; cross-judge-model agreement is $\kappa = 0.28$ overall and 0.55–0.68 on the hypothesis-critical fields: alternatives, unused branches, and redundant verification. Judge-sensitive fields such as task restatement and post-solution reasoning rely on deterministic proxies in the quantitative claims. Appendix G provides verbatim evidence examples.

D.3 Tool taxonomy and completion proxy

A second frozen rubric classifies navigation, search, read, edit, test, build, lint, git, environment, and recovery calls. Redundancy rules are conservative: duplicate normalized commands with no intervening edit, re-reads of unchanged files, and reruns of already-green test scopes. Unit tests freeze the extractor; blind hand-derived labels caught one TAP pass-detection bug, which was fixed before condition-aware analysis.

“Post-success” uses a recoverable visible-test proxy: the last edit and first fully green visible test. The moment at which hidden evaluators would have passed during a run is not observable. PI.DEV also lacks per-call timing, and local CPU, GPU, and memory telemetry were not recorded.

D.4 Distributional and causal limits

Tool counts are zero-inflated and heavy-tailed: 94% of runs have zero post-green repeat tests, with a maximum of six. H-T1 and H-T4 tool-layer effects are directional rather than paper-robust. Prompt variants are assigned within paired experimental blocks, but semantic mechanism levels are observed after treatment; mechanism-level cost curves therefore do not identify randomized mediator effects. Cost uses pinned catalog prices at collection time. Provider-side safety classifiers may contribute latency, but server-side classification, queueing, and generation time cannot be separated from exposed usage fields; absence of unexplained billing is not evidence that no classifier runs.

E Post-Registration Replications and Expansion

E.1 Kimi-K3 replication

Within a day of Kimi-K3 appearing on the provider, the authors froze a protocol covering six pilot tasks, four variants (`baseline`, `multiple_approaches`, `deep_thinking`, `bounded_efficiency`), 72 PI.DEV runs, and a 24-run Claude Code directional arm. Material difference was defined before results as a reasoning ratio differing by more than 30% from both Kimi-K2 comparators, a classification flip, or more than five percentage points of scope-compliant-success change.

The PI.DEV arm completed 72/72 at 100% success, with an 81% cache rebate. Kimi-K3’s baseline median is 55 reasoning tokens versus 351 and 195 for the K2 comparators. Multiple approaches reaches $16.6\times$ [10.7, 34.1] and deep thinking $14.8\times$ [4.1, 33.9] against that low baseline. Absolute multiple- approaches reasoning remains at or below K2-generation levels (625 versus 1,606 median), and baseline cost per compliant success is the lowest of the three generations despite $2.5\text{--}3.3\times$ unit prices. Bounded efficiency is $0.89\times$ with its confidence interval below 1. Effect directions and classifications are preserved.

E.2 First-party claude-sonnet-5 study

The first-party study reverses the original stack: both harnesses call the Anthropic API, and Claude Code runs natively without gateway translation or model aliasing. The frozen protocol contains nine variants, six pilot tasks, and two repetitions: 108 PI.DEV runs and 54 native-Claude Code runs. All 162 are valid.

The Anthropic API includes thinking in output tokens without reporting it separately. The preregistered primary metric is therefore total-output-token ratio, defined consistently on the open-model side. Multiple approaches is $2.70\times$ [2.14, 3.86] under PI.DEV and $2.52\times$ [1.73, 3.48] under native Claude Code. Verbose repetition, autonomy, and bounded efficiency remain approximately neutral ($0.93\text{--}1.02\times$ for bounded efficiency). Native-Claude Code baseline no-cache cost per compliant success is approximately $15\times$ PI.DEV’s. Maximum certainty reaches $4.13\times$ output [1.42, 8.57] and $2.7\times$ no-cache cost per success. Deep thinking is $1.25\text{--}1.30\times$. Scope compliance is 162/162.

E.3 Registered expansion

The expansion targets four evidence gaps: (a) paraphrase generalization through 25 frozen arms including four forms per condition, length-matched controls, and instruction-position variants; (b) repetition stability; (c) 24 new tasks including a long-validation family and a second frozen 16-task holdout; and (d) per-model precision. Planned scale is approximately 5,600 new valid runs and approximately 10,250 total. Failed replications are committed to prominent reporting.

At writing, the paraphrase and stability arms contain 2,105 valid runs of 2,106: three models $\times$ six tasks $\times$ 25 arms $\times$ three new repetitions, and two models $\times$ 16 tasks $\times$ nine variants $\times$ three new repetitions. The registered stability analysis reports within-cell no-cache cost CV over five-repetition cells. All four paraphrase forms of multiple approaches, deep thinking, and maximum certainty elevate reasoning on every model tested (12/12 condition-model cells, four of four forms). Length-matched controls are $0.96\text{--}1.05\times$. All five bounded- efficiency forms are $0.85\text{--}0.93\times$. Multiple-approaches magnitude ranges from $3.0\times$ for the concise form to $6.7\times$ for the verbose form; start versus end placement shifts an example effect from $5.2\times$ to $4.3\times$. Within-cell no-cache cost CV is 12–23%, and success is unanimous in 81–97% of cells. No effect-direction reversal is observed for a confirmed waste condition; reversal rate 0.33 appears only for verbose repetition

and no-questions autonomy, the two original neutral conditions. The new-task, second-holdout, and cross-harness expansion arms remain in progress.

F Exact Prompt Templates and Experimental Manipulations

All wording below is reproduced byte-exactly from the frozen prompt files used in the experiments (task `py-low-01` as the fixed representative rendering; placeholders `{OBJECTIVE}`, `{ACCEPTANCE_CRITERIA}`, `{TEST_COMMAND}` vary per task and are instantiated identically across variants of the same task). SHA-256 hashes of every displayed prompt match the hashes recorded in the run ledger. `<TURN-BREAK>` marks the protocol’s fixed multi-turn boundary.

F.1 Primary semantically controlled variants

baseline Family: semantically controlled; hypothesis: baseline; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 `e48b2d5b1f81` (ledger-verified). *Complete rendered prompt:*

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
```

verbose_repetition Family: semantically controlled; hypothesis: H1; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 `1f96a9bf4320` (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

```
To restate the requirements so nothing is missed: the change must stay within the
scope given above (shop/discount.py only), it must satisfy the same acceptance criteria
already
listed, and the verification command remains python3 -m unittest discover -s tests
-t . -v. In other words, do the
task exactly as described, within exactly the same boundaries, checked by exactly
the same tests, and finish under exactly the same stop condition as stated before.
```

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
To restate the requirements so nothing is missed: the change must stay within the
scope given above (shop/discount.py only), it must satisfy the same acceptance criteria
already
listed, and the verification command remains python3 -m unittest discover -s tests
-t . -v. In other words, do the
task exactly as described, within exactly the same boundaries, checked by exactly
the same tests, and finish under exactly the same stop condition as stated before.
```


`deep_thinking` Family: semantically controlled; hypothesis: H2; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 `d8be42655ded` (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

Think very deeply about this. Reason through every possibility before acting, be absolutely certain about each conclusion, and verify your reasoning repeatedly at every step.

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
Think very deeply about this. Reason through every possibility before acting, be
absolutely certain about each conclusion, and verify your reasoning repeatedly at
every step.
```

`exhaustive_exploration` Family: semantically controlled; hypothesis: H2; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 `7fa4f53c909e` (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

Before editing anything, inspect the entire repository and make sure you understand the complete system: read every source file and understand how all the parts connect, so your change is informed by the whole codebase.

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
Before editing anything, inspect the entire repository and make sure you understand
the complete system: read every source file and understand how all the parts
connect, so your change is informed by the whole codebase.
```

`multiple_approaches` Family: semantically controlled; hypothesis: H3; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 `820999071ae6` (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

Develop several distinct approaches to this fix, compare their trade-offs explicitly, and only then select and implement the best one.

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
```

passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.
Develop several distinct approaches to this fix, compare their trade-offs explicitly, and only then select and implement the best one.

`max_certainty` Family: semantically controlled; hypothesis: H4; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 1b35cdaf8727 (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

You must be absolutely confident in the result. Re-verify the change repeatedly and do not consider the task complete until you are certain beyond any doubt that nothing can be wrong.

Complete rendered prompt:

Fix the discount bug in `shop/discount.py`: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: `shop/discount.py` only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under `tests/passes`
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.
You must be absolutely confident in the result. Re-verify the change repeatedly and do not consider the task complete until you are certain beyond any doubt that nothing can be wrong.

`adjacent_cleanup` Family: semantically controlled; hypothesis: H5; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 58dddddcc1d43 (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

While you are in there, feel free to clean up anything adjacent that could be improved: refactor awkward code, tidy naming, and make related general improvements where you see the opportunity.

Complete rendered prompt:

Fix the discount bug in `shop/discount.py`: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: `shop/discount.py` only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under `tests/passes`
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.
While you are in there, feel free to clean up anything adjacent that could be improved: refactor awkward code, tidy naming, and make related general improvements where you see the opportunity.

`no_questions_autonomy` Family: semantically controlled; hypothesis: H5; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 95bb97b2dc91 (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

Do not ask me any questions. Infer any missing details yourself and do whatever is necessary to complete the task autonomously.

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
Do not ask me any questions. Infer any missing details yourself and do whatever
is
necessary to complete the task autonomously.
```

bounded_efficiency Family: semantically controlled; hypothesis: H6; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 56954fa9ae17 (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

```
Work efficiently: begin with the failing test and the most likely implementation
files; inspect additional files only when evidence requires it; avoid unrelated
cleanup; make the smallest sufficient change; run the relevant tests; stop as soon
as the acceptance criteria pass.
```

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
Work efficiently: begin with the failing test and the most likely implementation
files; inspect additional files only when evidence requires it; avoid unrelated
cleanup; make the smallest sufficient change; run the relevant tests; stop as soon
as the acceptance criteria pass.
```

F.2 Harness-axis variants

goal_only Family: semantically controlled; hypothesis: H12; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 9e72e502257b (ledger-verified). *Baseline text removed/replaced:*

```
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
```

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
```

scoped.authorization Family: semantically controlled; hypothesis: H13; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 362380e7ee68 (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

```
You are authorized to modify only:  shop/discount.py only
You are NOT authorized to modify anything else; treat all other paths as read-only.
```

Baseline text removed/replaced:

```
Scope:  shop/discount.py only
```

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py:  apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
You are authorized to modify only:  shop/discount.py only
You are NOT authorized to modify anything else; treat all other paths as read-only.
Acceptance criteria:  apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with:  python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
```

F.3 Stress variants (intentionally break semantic equivalence; analyzed separately)

misleading.architecture Family: stress; hypothesis: stress family; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 f01a269082e3 (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

```
Note:  we believe the problem is caused by the test fixtures in tests/ --- start
there.
```

Baseline text removed/replaced:

```
Scope:  shop/discount.py only
Stop when the tests pass.
```

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py:  apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Note:  we believe the problem is caused by the test fixtures in tests/ --- start
there.
Acceptance criteria:  apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with:  python3 -m unittest discover -s tests -t . -v
```

ambiguous.scope Family: stress; hypothesis: stress family; multi-turn: no; objective/criteria/test preserved verbatim: no (by design); SHA-256 37d1142ad44f (ledger-verified). *Text added relative to baseline (placement: prompt start):*

```
Something in this project is producing wrong results related to the following
area:  price and discount calculations.  Improve the situation.
```

Baseline text removed/replaced:

Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/ passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.

Complete rendered prompt:

Something in this project is producing wrong results related to the following area: price and discount calculations. Improve the situation.

conflicting_constraints Family: stress; hypothesis: stress family; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 3a8fab0b657 (ledger-verified). *Text added relative to baseline (placement: after the baseline task block):*

Be extremely thorough and examine everything carefully, but also finish as fast as possible and touch almost nothing. Do a complete, deep job quickly and minimally.

Baseline text removed/replaced:

Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/ passes
Stop when the tests pass.

Complete rendered prompt:

Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Be extremely thorough and examine everything carefully, but also finish as fast as possible and touch almost nothing. Do a complete, deep job quickly and minimally.
Verify with: python3 -m unittest discover -s tests -t . -v

irrelevant_context Family: stress; hypothesis: stress family; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 dbdf31b964ba (ledger-verified). *Text added relative to baseline (placement: prompt start):*

Background (for context): our organization has recently migrated from a monorepo to a polyrepo layout, then partially back, and several teams maintain conflicting style guides; the previous maintainer preferred functional patterns but the current team is split, and there is an ongoing debate about whether to adopt a plugin architecture in Q3, which the architecture council has deferred twice pending a review of the service mesh proposal that may or may not affect this component. None of that debate is resolved. Meanwhile, the actual request:

Complete rendered prompt:

Background (for context): our organization has recently migrated from a monorepo to a polyrepo layout, then partially back, and several teams maintain conflicting style guides; the previous maintainer preferred functional patterns but the current team is split, and there is an ongoing debate about whether to adopt a plugin architecture in Q3, which the architecture council has deferred twice pending a review of the service mesh proposal that may or may not affect this component.

None of that debate is resolved. Meanwhile, the actual request:
Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.

`missing_criteria` Family: stress; hypothesis: stress family; multi-turn: no; objective/criteria/test preserved verbatim: yes; SHA-256 9e72e502257b (frozen template; no executed run of this variant on this task – rendered for documentation). *Baseline text removed/replaced:*

Scope: shop/discount.py only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.

Complete rendered prompt:

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.

`split_across_turns` Family: stress; hypothesis: stress family; multi-turn: yes; objective/criteria/test preserved verbatim: no (by design); SHA-256 828663820120 (ledger-verified). *Text added relative to baseline (placement: prompt start):*

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the
<TURN-BREAK>
Also: price AFTER applying a pct-percent discount, not the discount amount itself.
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes. Verify with: `python3 -m unittest discover -s tests -t . -v`. Stop when the tests pass.

Baseline text removed/replaced:

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.

Complete rendered prompt:

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the
<TURN-BREAK>
Also: price AFTER applying a pct-percent discount, not the discount amount itself.
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes. Verify with: `python3 -m unittest discover -s tests -t . -v`. Stop when the tests pass.

full_restatement_per_turn Family: stress; hypothesis: stress family; multi-turn: yes; objective/criteria/test preserved verbatim: yes; SHA-256 53f74e6609c7 (ledger-verified). *Text added relative to baseline (placement: prompt start):*

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only Acceptance criteria: apply_discount(100, 20) == 80.0
and every test under tests/ passes Verify with: python3 -m unittest discover -s
tests -t . -v Stop when the tests pass.
```

<TURN-BREAK>

```
Reminder of the full task, restated completely: Fix the discount bug in shop/discount.py:
apply_discount(price, pct) must return the price AFTER applying a pct-percent discount,
not the discount amount itself. Scope: shop/discount.py only Acceptance criteria:
apply_discount(100, 20) == 80.0 and every test under tests/ passes Verify with: python3
-m unittest discover -s tests -t . -v Stop when the tests pass. Is it done? If
not, continue.
```

Baseline text removed/replaced:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
```

Complete rendered prompt:

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only Acceptance criteria: apply_discount(100, 20) == 80.0
and every test under tests/ passes Verify with: python3 -m unittest discover -s
tests -t . -v Stop when the tests pass.
```

<TURN-BREAK>

```
Reminder of the full task, restated completely: Fix the discount bug in shop/discount.py:
apply_discount(price, pct) must return the price AFTER applying a pct-percent discount,
not the discount amount itself. Scope: shop/discount.py only Acceptance criteria:
apply_discount(100, 20) == 80.0 and every test under tests/ passes Verify with: python3
-m unittest discover -s tests -t . -v Stop when the tests pass. Is it done? If
not, continue.
```

F.4 Complete before-and-after example (py-low-01)

Text in **bold** differs from the baseline rendering; everything else is byte-identical.

baseline

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
Stop when the tests pass.
```

multiple_approaches

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.
Develop several distinct approaches to this fix, compare their trade-offs explicitly, and only then select and implement the best one.

deep_thinking

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.
Think very deeply about this. Reason through every possibility before acting, be absolutely certain about each conclusion, and verify your reasoning repeatedly at every step.

max_certainty

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.
You must be absolutely confident in the result. Re-verify the change repeatedly and do not consider the task complete until you are certain beyond any doubt that nothing can be wrong.

bounded_efficiency

Fix the discount bug in shop/discount.py: `apply_discount(price, pct)` must return the price AFTER applying a pct-percent discount, not the discount amount itself.
Scope: shop/discount.py only
Acceptance criteria: `apply_discount(100, 20) == 80.0` and every test under tests/passes
Verify with: `python3 -m unittest discover -s tests -t . -v`
Stop when the tests pass.
Work efficiently: begin with the failing test and the most likely implementation files; inspect additional files only when evidence requires it; avoid unrelated cleanup; make the smallest sufficient change; run the relevant tests; stop as soon as the acceptance criteria pass.

`misleading.architecture`

```
Fix the discount bug in shop/discount.py: apply_discount(price, pct) must return
the price AFTER applying a pct-percent discount, not the discount amount itself.
Note: we believe the problem is caused by the test fixtures in tests/ --- start
there.
Acceptance criteria: apply_discount(100, 20) == 80.0 and every test under tests/
passes
Verify with: python3 -m unittest discover -s tests -t . -v
```

G Annotated Reasoning-Trace Examples

Each example below is a REAL excerpt from a recorded reasoning trace in the corpus, located by the condition-blind judge's mandatory evidence quote for the named mechanism and re-verified verbatim against the raw trace file. Excerpts are shown with minimal surrounding context; run identifiers give model, task, and condition. These illustrate what the mechanism counts in Section 3 actually refer to.

Unused solution branches H-S1: branches elaborated then discarded; only one is implemented. Run: DeepSeek-V4-Pro, task `go-high-01`, condition `multiple_approaches`, turn 3. *Judge evidence quote (verbatim from the recorded trace):*

```
sync.Map is a bit awkward for counters. Let me reconsider.
```

Surrounding trace context:

```
.counts.LoadOrStore(key, new(int)) p := v.(*int) atomic.AddInt32(p, 1) // types don't match... } ``
Actually sync.Map is a bit awkward for counters. Let me reconsider. **Trade-offs:** - [U+274C] 'sync.Map'
doesn't have a simple atomic increment - [U+274C] Need CAS loop or store pointers - [U+274C] Complex,
error-prone - [U+274C] 'Get' returns 'interface{}' - [U+274C] Not a good fi...
```

Redundant verification H-S3: re-establishing an already-settled fact with no new evidence. Run: GLM-5.2, task `py-low-02`, condition `max_certainty`, turn 4. *Judge evidence quote (verbatim from the recorded trace):*

```
Tests pass. Let me verify all acceptance criteria explicitly and re-read the file
to confirm.
```

Surrounding trace context:

```
Tests pass. Let me verify all acceptance criteria explicitly and re-read the file to confirm....
```

Post-solution reasoning H-S3: deliberation after the last edit and a fully-green test. Run: GLM-5.2, task `py-med-02`, condition `max_certainty`, turn 6. *Judge evidence quote (verbatim from the recorded trace):*

```
All 3 tests pass. Now let me thoroughly verify all the acceptance criteria mentioned
in the task, beyond just the existing tests, to be "absolutely confident".
```

Surrounding trace context:

```
All 3 tests pass. Now let me thoroughly verify all the acceptance criteria mentioned in the task, beyond
just the existing tests, to be "absolutely confident". Acceptance criteria from the task: 1. 'parse_value('true')'
is True 2. 'parse_v...
```

Unsupported assumptions H-S4: claims never grounded in inspected code (induced by a wrong hint). Run: Kimi-K2.6, task py-med-01, condition misleading_architecture, turn 0. *Judge evidence quote (verbatim from the recorded trace):*

Problem believed to be caused by rounding helpers in cart/pricing.py.

Surrounding trace context:

```
mount must be deducted from subtotal BEFORE tax is applied. The docstring says that. Current code
deducts it after tax. Problem believed to be caused by rounding helpers in cart/pricing.py. We need
to inspect files, understand current code, then fix. We'll read the files: cart/checkout.py, cart/pricing.py,
and tests. We need to verify tests: total(100,10) ...
```

Task restatement / elaboration H-S2 context: repetition inside deep-thinking traces (judge-weak field; shown for illustration, quantified via deterministic proxies). Run: Kimi-K2.6, task js-low-03, condition deep_thinking, turn 11. *Judge evidence quote (verbatim from the recorded trace):*

The user wants me to stop when the tests pass. Let me wait for the test output.

Surrounding trace context:

The user wants me to stop when the tests pass. Let me wait for the test output....